

Trees

Tree functions ఎప్పుడూ **ఏదో ఒకటి** తిరిగి ఇస్తాయి. మరియు ఆ "ఏదో ఒకటి" ఎప్పుడూ ఒక జవాబులాగే కనిపిస్తుంది.

మొదటిది – "ఇది BST నా?" అనే సహజమైన తనిఖీ యాదృచ్ఛిక trees మీద **1.38%** మాత్రమే తప్పు, మరియు నిజంగా విరిగిన trees మీద **35.4%**. రెండోది – sorted data ని BST lo చేరిస్తే అది tree కాదు, **linked list**: నిర్మాణం **328x**, వెతుకులాట **307x**, ఆపై **RangeError**. మూడోది – పాఠ్యపుస్తక LCA రెండు nodes ఉంటే **0.0%** తప్పు, ఒకటి లేకపోతే **100.0%**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే – §4 యొక్క **1.38% vs 35.4%** ఒక code మీద ఒక పరీక్ష వేర్వేరు హారంతో (denominator).

TREES ఈ SERIES కి ఒక ముగింపు – మూడు విరుపులూ ఒకే ఆకారం

Deep 09 lo తప్పు comparator sort ని విఫలం చేయలేదు; అది sort ని అర్థరహితం చేసింది.

Trees lo అదే, ఇంకా తీవ్రంగా: **మూడు bugs, మూడూ ఒక node ని (లేదా true ని) ఆత్మవిశ్వాసంతో తిరిగి ఇస్తాయి.** ఒక్కటే crash కాదు, ఒక్కటే null ఇవ్వదు, ఒక్కటే "నాకు తెలియదు" అనదు.

కాబట్టి ప్రశ్న ఎప్పుడూ ఒకటే: **"ఈ function ఏ ఊహ మీద నిలబడి ఉంది, మరియు ఆ ఊహ తప్పుతే అది నాకు చెబుతుందా?"**

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Tree ఒక పునరావృత నిర్వచనం
2. అడగాల్సిన ప్రశ్నలు

Part 2 – మొదటి విరుపు: స్థానిక తనిఖీ

3. Step – "ఇది BST నా?"
4. **మొదటి విరుపు** – 1.38%, ఆపై 35.4%
5. Step – పరిధిని కిందికి పంపడం

Part 3 – రెండో విరుపు: ఆకారం

6. Step – BST కట్టడం
7. **రెండో విరుపు** – sorted data ఇస్తే tree కాదు, జాబితా
8. Step – recursion ని తీసేయడం

Part 4 – మూడో విరుపు: LCA

9. Step – పాఠ్యపుస్తక LCA
10. **మూడో విరుపు** – 0.0% మరియు 100.0%
11. Step – ఊహని ఒక పంక్తిగా రాయడం

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code
13. 21,000 trees · mutation testing

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Tree ఒక పునరావృత నిర్వచనం

Tree ఒక data structure కంటే ముందు ఒక నిర్వచనం: "ఒక tree అంటే ఒక విలువ, మరియు రెండు trees."

అందుకే tree code చిన్నగా ఉంటుంది – నిర్వచనాన్ని అలాగే రాస్తే సరిపోతుంది. మరియు అందుకే **అది తప్పినప్పుడు కూడా చిన్నగానే ఉంటుంది.**

TREES LO రెండు వేర్వేరు రకాల నియమాలు ఉంటాయి

స్థానిక నియమం (local) – ఒక node ని మరియు దాని పిల్లలని చూస్తే చాలు. ఉదా: "ఈ node కి రెండు కంటే ఎక్కువ పిల్లలు లేరు."

సార్వత్రిక నియమం (global) – పైన ఉన్న అన్ని పూర్వకులని కూడా చూడాలి. ఉదా: "ఇది BST".

BST నిర్వచనం "ఎడమ పిల్ల చిన్నది" కాదు. అది "ఎడమ ఉపవృక్షం మొత్తం చిన్నది".

§4 lo చూస్తూ: ఈ ఒక్క పదం – "ఉపవృక్షం" – మార్చితే, విరిగిన trees lo 35.4% పట్టుబడవు.

2. అడగాల్సిన ప్రశ్నలు

ప్రశ్న	ఎందుకు అడుగుతున్నాం
నకళ్ళు ఉంటాయా?	ఉంటే BST నిర్వచనమే మారుతుంది – <code><</code> ఆ, <code><=</code> ఆ? §13 lo ఒక mutation ఇదే
Data ఏ క్రమంలో వస్తుంది?	§7 – sorted గా వస్తే మీ BST ఒక linked list
Tree ఎంత లోతు ఉండొచ్చు?	§8 – 10,000 దాటితే recursive traversal <code>RangeError</code>
Node లేకపోతే ఏమి తిరిగి ఇవ్వాలి?	§10 – ఇది అడగకపోతే మీ LCA ఒక తప్పు node ఇస్తుంది
Balanced గా ఉంటాలా?	ఉంచాలంటే AVL/Red-Black, లేదా ముందే కలిపి చేర్చడం (§7)

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Nodes కి parent pointer ఉందా?	ఉంటే LCA చాలా సులభం – రెండు జాబితాలని పైకి నడిపితే చాలు
విలువలు ప్రత్యేకమైనవా?	కాకపోతే "LCA" అనేదే అస్పష్టం – ఏ 5 గురించి మాట్లాడుతున్నాం?

ఒక ప్రశ్న అడిగితే మరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"node tree lo లేకపోతే ఏమి తిరిగి ఇవ్వాలి?"

ఎందుకంటే §10 lo కొలిచినది ఇదే: రెండు nodes ఉంటే పాఠ్యపుస్తక LCA **10,000 cases lo 0 తప్పులు**. ఒకటి లేకపోతే **10,000 lo 10,000 తప్పు**.

మరియు అది `null` ఇవ్వదు – అది **రెండో node ని** ఇస్తుంది. ఒక పూర్తిగా సహేతుకంగా కనిపించే జవాబు.

Part 2 – మొదటి విరుపు: స్థానిక తనిఖీ

3. Step – "ఇది BST నా?"

నిర్వచనం నేరుగా రాద్ధాం:

```
function isBST(t) {  
    if (!t) return true;  
    if (t.l && t.l.v >= t.v) return false;    // ఎడమ పిల్ల చిన్నదై ఉండాలి  
    if (t.r && t.r.v <= t.v) return false;    // కుడి పిల్ల పెద్దదై ఉండాలి  
    return isBST(t.l) && isBST(t.r);  
}
```

ఐదు పంక్తులు, నిర్వచనాన్ని మక్కికి మక్కి. ఒక BST ఇస్తే `true`, ఒక విరిగిన tree ఇస్తే `false` – పరీక్షించాను, పనిచేసింది.

4. మొదటి విరుపు – 1.38%, ఆపై 35.4%

isBST – "ప్రతి node ని తన పిల్లలతో పోల్చు" అనే తనిఖీ

data ఆకారం	trees	local తప్పు	%
పూర్తిగా యాదృచ్ఛిక trees	20,000	275	1.38%
ఒక node విలువ ± 5 మార్పితే	20,000	1,265	6.33%
ఒక node విలువ ± 50 మార్పితే	20,000	3,605	18.02%

1.38%. ఒక్క శాతం. "దాదాపు ఎప్పుడూ సరైనదే" అనిపిస్తుంది. కానీ ఆ హారం (denominator) తప్పు – చాలా trees అస్సలు BST కావు, మరియు అవి మొదటి పోలిక దగ్గరే దొరుకుతాయి. నిజమైన ప్రశ్న: విరిగిన trees లో ఎన్ని తప్పించుకున్నాయి?

నిజంగా BST కాని trees : 14,670
అందులో local తనిఖీ "BST నే" అన్నది : 5,186 (35.4%)

విరిగిన ప్రతి మూడు TREES LO ఒకటి తప్పించుకుంటోంది

1.38% మరియు 35.4% – ఇవి ఒకే code, ఒకే పరీక్ష. తేడా హారంలో మాత్రమే.

1.38% "అన్ని trees lo" · 35.4% "ఈ function పట్టుకోవాల్సిన trees lo".

రెండేదే function యొక్క పని. మరియు అక్కడ అది మూడింట ఒక వంతు విఫలం.

అతి చిన్న counterexample:

```
      10
     /  \
    5    15
   /  \
  1    12
```

← 12 > 10, కానీ అది 10 కి *ఎడమ* వైపు ఉంది

local తనిఖీ : true ← 12 > 5 ✓, మరియు అంతకంటే పైకి చూడదు
నిజం : false
in-order : 1 5 12 10 15 ← 12 తర్వాత 10 – పెరగడం లేదు

"ప్రతి NODE సరైనదే. TREE తప్పు."

12 తన తల్లి 5 కంటే పెద్దది – సరైనదే. 5 తన తల్లి 10 కంటే చిన్నది – సరైనదే. ప్రతి స్థానిక నియమమూ పాటించబడింది.

కానీ 12 10 యొక్క ఎడమ ఉపవృక్షంలో ఉంది, మరియు అక్కడ ఉన్న ప్రతిదీ 10 కంటే చిన్నదై ఉండాలి.

ఆ నియమాన్ని ఉల్లంఘిస్తున్న node ఆ నియమాన్ని విధించే node ని చూడలేదు – అవి tree lo రెండు అంతస్తుల దూరంలో ఉన్నాయి.

ఇదే తేడా: స్థానికంగా సరైనది ≠ సార్వత్రికంగా సరైనది.

5. Step – పరిధిని కిందికి పంపడం

పరిష్కారం: ప్రతి node కి ఒక చెల్లుబాటు కీటికీ (lo, hi) పంపడం, మరియు కిందికి వెళ్ళే కొద్దీ అది ఇరుకవడం.

```
function isBST(root) {
  const stack = [[root, -Infinity, Infinity]];
  while (stack.length) {
    const [t, lo, hi] = stack.pop();
    if (!t) continue;
    if (!(t.v > lo && t.v < hi)) return false;
    stack.push([t.l, lo, t.v], [t.r, t.v, hi]); // ← ఇక్కడ కీటికీ ఇరుకవుతుంది
  }
  return true;
}
```

ఆ చివరి పంక్తీ మొత్తం తేడా. 12 కి ఇప్పుడు కిటికీ (5, 10) – 10 అనే హద్దు రెండు అంతస్తుల పైనుంచి కిందికి ప్రయాణించి వచ్చింది.

§13 lo: ఆ పంక్తిని `stack.push([t.l, lo, hi], [t.r, lo, hi])` గా మార్చితే (కిటికీ ఇరుకవ్వకపోతే) – **120/120 విఫలం.**

Part 3 – రెండో విరుపు: ఆకారం

6. Step – BST కట్టడం

```
let root = null;
for (const v of values) root = insert(root, v);
```

`insert` సరైనది, `isBST` నెగ్గుతుంది, `find` పనిచేస్తుంది. ప్రతి జవాబూ ఖచ్చితంగా సరైనది.

ఇప్పుడు `values` ఎక్కడి నుంచి వస్తాయి? తరచుగా – ఒక database నుంచి, ఒక file నుంచి, ఒక API నుంచి. అంటే తరచుగా అవి ఇప్పటికే sorted.

7. రెండో విరుపు – sorted data ఇస్తే tree కాదు, జాబితా

BST · 1 నుంచి n వరకు విలువలు – ఏ క్రమంలో చేర్చామో దాన్ని బట్టి

	n	sorted గా చేర్చితే	కలిపి చేర్చితే	$\log_2 n$	1,000 వెతుకుళ్ళు (sorted / కలిపి)
-----	-----	-----	-----	-----	-----
6x	100	100	17	7	51,410 / 8,528 →
45x	1000	1,000	20	10	5,01,187 / 11,261 →
180x	5000	5,000	24	13	25,58,787 / 14,252 →
307x	10000	10,000	30	14	50,39,342 / 16,405 →

లోతు = N. సరిగ్గా N. ఒక్క కొమ్మ కూడా లేదు.

1, 2, 3, ... అని చేర్చితే ప్రతి కొత్త విలువ అన్నిటికంటే పెద్దది. కాబట్టి అది ఎప్పుడూ కుడి వైపు వెళ్తుంది.

మీరు కట్టింది tree కాదు – ఒక linked list, tree యొక్క ఖరీదుతో.

మరియు ప్రతి జవాబూ ఇప్పటికే ఖచ్చితంగా సరైనది. `isBST` true అంటుంది. `find` సరైన జవాబే ఇస్తుంది. ఒక్క పరీక్షా విఫలం కాదు.

నిర్మాణ ఖరీదు · $n = 10,000$

sorted గా చేర్చితే : 4,99,95,000 పోలికలు ($n^2/2 = 5,00,00,000$)

కలిపి చేర్చితే : 1,52,476 పోలికలు (n log₂n = 1,32,877)
రెట్లు : 328x

4,99,95,000 మరియు $n^2/2 = 5,00,00,000$. ఇది "దాదాపు $O(n^2)$ " కాదు – ఇది సరిగ్గా $n^2/2$.

recursive in-order traversal · default stack

n = 1,000 (sorted గా చేర్చినది) | సరే · 1000 nodes
n = 2,000 (sorted గా చేర్చినది) | సరే · 2000 nodes
n = 5,000 (sorted గా చేర్చినది) | సరే · 5000 nodes
n = 10,000 (sorted గా చేర్చినది) | RangeError: Maximum call stack size exceeded

పరిష్కారం ఒక పంక్తి – కలిపి చేర్చడం

లోతు 10,000 నుంచి **30** కి ($\log_2 10,000 = 14$, మరియు యాదృచ్ఛిక BST సగటున ~2 రెట్లు లోతు).

Deep 09 lo quicksort sorted input మీద crash అయింది · **Deep 06** lo quickselect sorted input మీద 1,189x · ఇప్పుడు BST.

"Sorted input" ఒక సులభమైన సందర్భం కాదు. చాలా structures కి అదే అత్యంత చెడ్డ సందర్భం – ఎందుకంటే అవన్నీ "data కలిపి ఉంటుంది" అనే ప్రకటించని ఊహ మీద నిలబడతాయి.

8. Step – recursion ని తీసేయడం

RangeError కి నిజమైన పరిష్కారం tree ని సమతుల్యం చేయడం. కానీ **మే చేతిలో లేని data** వస్తే? అప్పుడు traversal నే recursion లేకుండా రాయాలి:

```
function inorder(root) {  
  const out = [], stack = [];  
  let t = root;  
  while (t || stack.length) {  
    while (t) { stack.push(t); t = t.l; } // ఎడమకి దిగినంత దిగు  
    t = stack.pop();  
    out.push(t.v); // ఆపై ఈ node  
    t = t.r; // ఆపై కుడి వైపు  
  }  
  return out;  
}
```

ఇది అదే పనిని **మన స్వంత array** మీద చేస్తుంది, JS call stack మీద కాదు. Array లక్షల మూలకాలు పడుతుంది; call stack ~10,000 దగ్గర ఆగిపోతుంది.

§13 lo: `out.push(t.v)` ని ఎడమకి దిగే ముందు పెట్టితే అది pre-order అవుతుంది – **120/120** విఫలం.

Part 4 – మూడో విరుపు: LCA

9. Step – పాఠ్యపుస్తక LCA

Lowest Common Ancestor – రెండు nodes కీ ఉమ్మడిగా ఉన్న అత్యంత కింది పూర్వీకుడు:

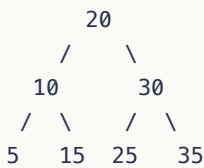
```
function lca(root, p, q) {  
  if (!root || root.v === p || root.v === q) return root;  
  const L = lca(root.l, p, q), R = lca(root.r, p, q);  
  if (L && R) return root;      // p ఒక వైపు, q ఇంకో వైపు → ఇదే LCA  
  return L || R;                // రెండూ ఒకే వైపు → ఆ వైపు జవాబే  
}
```

నాలుగు పంక్తులు. ఇది చాలా చక్కని code – మరియు ఇది సరైనదే.

10. మూడో విరుపు – 0.0% మరియు 100.0%

LCA – ఒక node tree lo లేకపోతే ఏమవుతుంది?

దృశ్యం	cases	పాఠ్యపుస్తక LCA తప్పు	%
రెండు nodes ఉన్నాయి	10,000	0	0.0%
ఒక node లేదు	10,000	10,000	100.0%
రెండూ లేవు	10,000	0	0.0%



`lca(5, 99) → 5` ← 99 ఈ tree lo లేనే లేదు
నిజం → null

0.0% మరియు 100.0% - మధ్యలో ఏమీ లేదు

ఇది "కొన్నిసార్లు తప్పు" కాదు. ఇది **రెండు వేర్వేరు functions**: ఊహ నిజమైతే ఖచ్చితంగా సరైనది, ఊహ తప్పుతే ఖచ్చితంగా తప్పు.

కారణం ఆ మొదటి పంక్తి: `if (root.v === p || root.v === q) return root`. అది "p ని కనుగొన్నాను" అని అర్థం చేసుకుంటుంది. కానీ నిజానికి అది "p ఇక్కడ ఉంది, మరియు q ఎక్కడో ఉందని నేను నమ్ముతున్నాను".

q లేకపోతే - recursion ఇంకో వైపు `null` తెస్తుంది, `L || R` p నే తిరిగి ఇస్తుంది, మరియు ఆ జవాబు **ఒక చెల్లుబాటు అయ్యే node**.

"రెండూ లేవు" అన్నప్పుడు 0.0% - ఎందుకంటే అప్పుడు అది సరైన కారణంతో కాదు, ప్రమాదవశాత్తు `null` ఇస్తుంది.

11. Step - ఊహని ఒక పంక్తిగా రాయడం

పరిష్కారం తెలివైనది కానక్కర్లేదు. ప్రకటించని ఊహని ఒక పంక్తిగా రాయడమే సరిపోతుంది:

```
function lca(root, p, q) {  
  if (!find(root, p) || !find(root, q)) return null; // ← ఊహ, ఇప్పుడు ఒక తనిఖీ  
  const go = (t) => { ... }; // మిగతాది యథాతథం  
  return go(root);  
}
```

ఖరీదు: రెండు అదనపు వెతుకుళ్ళు, అంటే **O(h)** - మొత్తం **O(n)** తో పోలిస్తే ఏమీ కాదు.

ఇదే ఈ **SERIES** యొక్క కేంద్ర పద్ధతి

Deep 01 - "predicate ఏకదిశ" · **Deep 05** - "బరువులు ≥ 0 " · **Deep 08** - "greedy ఎంపికని OPT lo పెట్టొచ్చు" · ఇప్పుడు - "p మరియు q tree lo ఉన్నాయి".

ప్రతినారీ అదే రెండు అడుగులు: **(1)** ఊహ ఏమిటో పదార్థోరాయడం · **(2)** దాన్ని ఒక assertion గా, లేదా ఒక తనిఖీగా, లేదా ఒక fuzz నియమంగా మార్చడం.

రాయలేని ఊహ పరీక్షించలేని ఊహ. మరియు పరీక్షించలేని ఊహ ఒక రోజు మీ **production data** ని తినేస్తుంది.

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code

```
// BST నా? – *పరిధి* తే, పిల్లలతో కాదు
function isBST(root) {
  const stack = [[root, -Infinity, Infinity]];
  while (stack.length) {
    const [t, lo, hi] = stack.pop();
    if (!t) continue;
    if (!(t.v > lo && t.v < hi)) return false;
    stack.push([t.l, lo, t.v], [t.r, t.v, hi]);
  }
  return true;
}
```

```
// సమతుల్యమా? – post-order, ప్రతి node దగ్గర ఎత్తు *మళ్ళీ* లెక్కించకుండా
const h = new Map([[null, 0]]);
const stack = [[root, false]];
while (stack.length) {
  const [t, ready] = stack.pop();
  if (!t) continue;
  if (!ready) { stack.push([t, true], [t.l, false], [t.r, false]); continue; }
  const a = h.get(t.l) ?? 0, b = h.get(t.r) ?? 0;
  if (Math.abs(a - b) > 1) return false;
  h.set(t, Math.max(a, b) + 1);
}
```

ఆ **ready** గుర్తు ముఖ్యం: ఒక node ని **రెండుసార్లు** stack మీద పెడతాం – ఒకసారి "పిల్లలని చెయ్యి", ఇంకోసారి "ఇప్పుడు నన్ను తేల్చు". దీనితో ఎత్తు ప్రతి node కి **ఒక్కసారే** లెక్కుతుంది – పాఠ్యపుస్తక రూపం $O(n^2)$, ఇది **$O(n)$** .

మొత్తం module lo recursion ఒకే ఒక్క చోట – **lca** యొక్క లోపలి **go**, మరియు అది **§11 యొక్క తనిఖీ నెగ్గిన తర్వాతే** నడుస్తుంది.

13. 21,000 trees • mutation testing

పద్నాలుగు నియమాలు: **isBST = in-order** పెరుగుదల, కట్టిన **BST** నిజంగా **BST**, **inorder = సూచన**, **BST** యొక్క **inorder sorted**, చేర్చినవన్నీ దొరుకుతాయి, లేనివి దొరకవు, **height = సూచన**, **find** అడుగులు $\leq$ ఎత్తు, **isBalanced = సూచన**, **lca = path-ఆధారిత సూచన**, **node** లేకపోతే **lca** తప్పక **null**, స్థానికంగా

సరైన trees మీద isBST = సూచన, ఏ ఆకారం మీదైనా isBST = సూచన, BST కాకపోయినా height/isBalanced/inorder సరైనవే, ఏ ఆకారం మీదైనా lca = సూచన.

ప్రతి సూచన (reference) పరీక్షించబడుతున్న code మీద ఆధారపడదు – Deep 08 యొక్క వృత్తాకార నియమం పాఠం.

300 runs · 7,500 BSTs · 6,000 స్థానికంగా-సరైన trees · 7,500 ఏ ఆకారపు trees అయినా నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

bsts	7,500	
arbitrary	7,500	
brokenBsts	6,611	← నిజంగా BST కాని trees
localOk	6,000	← స్థానిక తనిఖీ *నిర్మాణపరంగా* నెగ్గేవి
localOkBroken	3,806	← అందులో నిజంగా BST కానివి (63%)
balanced	879	
unbalanced	6,621	
lcaBoth	7,118	
lcaMissing	7,118	← §10 యొక్క దృశ్యం
deep	1,877	← sorted గా చేర్చినవి
dups	2,982	← నకళ్ళతో చేర్చినవి

==== mutation testing ====

మార్పు లేని code	→ 0/300 విఫలం
isBST lo పరిధికి బదులు పిల్లలతో పోలిక (§4 యొక్క bug)	→ 120/120 విఫలం
ఉదా: inorder 473 477 467 476 478 488 500	
isBST lo పరిధి కిందికి ఇరుకవ్వకపోతే	→ 120/120 విఫలం
ఉదా: inorder 473 477 467 476 478 488 500	
isBST lo `> lo && < hi` ని `>= lo && <= hi` గా	→ 53/120 విఫలం
ఉదా: isBST true ≠ false · inorder 1 1 8 8	
lca lo "రెండూ ఉన్నాయా" తనిఖీ తీసేస్తే (§10 యొక్క bug)	→ 120/120 విఫలం
ఉదా: lca(6, లేని 5073) = 6, null కావాలి	
lca lo `if (L && R) return t` తీసేస్తే	→ 120/120 విఫలం
ఉదా: lca(0,15) = 0, సూచన 14	
inorder lo విలువని ఎడమ వైపు దిగే *ముందు* push చేస్తే	→ 120/120 విఫలం
isBalanced lo `> 1` ని `> 2` గా	→ 120/120 విఫలం
height lo ఎడమ పిల్లకి `d + 1` కి బదులు `d`	→ 120/120 విఫలం
insert lo `v < t.v` ని `v <= t.v` గా	→ 120/120 విఫలం
ఉదా: insert BST ని విరగొట్టింది · n=7	

మొదటి ప్రయత్నంలో ఈ పట్టిక ఇంత బాగా లేదు

నా మొదటి fuzz తో మూడు ఫలితాలు బలహీనంగా వచ్చాయి:

§4 యొక్క bug – 35/120. అంటే నా యాదృచ్ఛిక trees ఆ bug ని **70% runs lo చూడనే లేదు**. ఎందుకు? §4 lo కొలిచిన కారణమే – చాలా విరిగిన trees **root దగ్గరే** విరిగి ఉంటాయి, మరియు స్థానిక తనిఖీ వాటిని పట్టుకుంటుంది.

insert lo v <= t.v – 0/120. నేను విలువలని new Set తో చేర్చాను, కాబట్టి నకలు అనే పరిస్థితే రాలేదు. ఆ mutation ఒక చేరుకోలేని దారి.

>= / <= – 9/120. అదే కారణం.

పరిష్కారం CODE LO కాదు – DATA LO

(1) ఒక కొత్త corpus: ఆకారం యాదృచ్ఛికం, విలువలు **తల్లి-పిల్ల నియమాన్ని మాత్రమే** పాటిస్తూ కేటాయించినవి. అంటే స్థానిక తనిఖీ **నిర్మాణపరంగా ఎప్పుడూ true**, మరియు trees lo **63%** నిజంగా BST కాదు. → **35/120 నుంచి 120/120.**

(2) 40% runs lo నకళ్ళతో చేర్చడం (విలువలు 0-24). → **0/120 నుంచి 120/120.**

>= / <= mutation 53/120 దగ్గర ఆగింది, మరియు అది **నిజాయితీగా ఆగింది**: సరైన insert నకలుని చేర్చదు, కాబట్టి ఒక BST lo రెండు సమాన విలువలు ఎప్పటికీ ఉండవు. ఆ mutation ని కేవలం **ఏ ఆకారమైనా** అనే corpus మాత్రమే చేరుకోగలదు, మరియు అందులో నకళ్ళు 120 runs lo 53 సార్లు కలిసి వచ్చాయి.

ఇది ఈ series lo ఆరోసారి: బలహీనమైన mutation ఫలితం అంటే బలహీనమైన code కాదు – బలహీనమైన data.

14. Follow-ups – balancing, Morris, tries

"Tree ని ఎప్పుడూ సమతుల్యంగా ఎలా ఉంచడం?"

AVL లేదా **Red-Black tree** – ప్రతి చేర్పు తర్వాత తిప్పడాల (rotations) ద్వారా ఎత్తును $O(\log n)$ lo ఉంచుతాయి. ఖరీదు: ప్రతి node కి అదనపు స్థితి (ఎత్తు లేదా రంగు), మరియు చాలా ఎక్కువ code. **నేను వీటిని కొలవలేదు** – §7 lo నేను కొలిచిన పరిష్కారం data ని కలిపి చేర్చడం, మరియు అది లోతును 10,000 నుంచి 30 కి తెచ్చింది.

"Traversal కి stack కూడా వద్దంటే?"

Morris traversal – tree lo తాత్కాలికంగా "threads" (కుడి pointers) కట్టి, తిరిగి వచ్చాక విప్పడం. **$O(1)$ memory.** ఖరీదు: ఇది tree ని తాత్కాలికంగా మార్పుతుంది, కాబట్టి ఒకసారి రెండు threads traverse చేయలేవు. **నేను దీన్ని కొలవలేదు.**

"Strings కోసం tree?"

Trie – ప్రతి అంచు ఒక అక్షరం. LLD Deep 17 lo నిజమైన 2,34,428-పదాల నిఘంటువు మీద దీన్ని కొలిచాను.

"Range queries కీ?"

Segment tree / Fenwick tree – "5 నుంచి 200 వరకు మొత్తం ఎంత?" అనే ప్రశ్నకి $O(\log n)$. LLD Deep 13 (Leaderboard) lo ఇదే సమస్య. ఈ doc lo నేను వేటిని కొలవలేదు.

"ఈ doc lo నేను ఏమి కొలవలేదు"

నాలుగు: **AVL/Red-Black, Morris traversal, segment trees**, మరియు **n-ary trees** – ఇక్కడ అంతా binary.

15. ఏమి నేర్చుకున్నాం

1. ప్రతి node సరైనదే కావచ్చు, tree తప్పు కావచ్చు. BST నిర్వచనం పేల్లల గురించి కాదు, ఉపవృక్షాల గురించి.

2. శాతం చూసినప్పుడు హోరం అడగండి. ఒకే bug – అన్ని trees lo **1.38%**, విరిగిన trees lo **35.4%**. రెండోదే function యొక్క పని.

3. పరిధిని కిందికి పంపడమే పరిష్కారం. (lo, hi) కిటికీ ప్రతి అంతస్తులో ఇరుకవుతుంది, మరియు అదే "ఉపవృక్షం" అనే పదాన్ని code lo రాయడం.

4. Sorted data ఇస్తే BST ఒక linked list. లోతు సరిగ్గా n, నిర్మాణం సరిగ్గా $n^2/2$ పోలికలు, వెతుకులాట **307x**.

5. మరియు ప్రతి జవాబూ ఇప్పటికీ సరైనదే. isBST true అంటుంది, find పనిచేస్తుంది, ఒక్క పరీక్షా విఫలం కాదు – కేవలం 328x ఖరీదు.

6. Sorted input ఒక సులభమైన సందర్భం కాదు. Deep 06, Deep 09, ఇప్పుడు Deep 10 – మూడింటికీ అదే అత్యంత చెడ్డ సందర్భం.

7. లోతు 10,000 దాటితే recursive traversal RangeError . Iterative రూపం మన స్వంత array వాడుతుంది, మరియు అది లక్షలు పడుతుంది.

8. LCA రెండు nodes ఉంటే 0.0% తప్పు, ఒకటి లేకపోతే 100.0%. మరియు అది null ఇవ్వదు – రెండో node ని ఇస్తుంది.

9. ప్రకటించని ఊహని ఒక పంక్తిగా రాయండి. if (!find(p) || !find(q)) return null – ఖరీదు $O(h)$, విలువ 100%.

10. Post-order + ఒక ready గుర్తు = $O(n^2)$ నుంచి $O(n)$. ఎత్తును ప్రతి node కి ఒక్కసారే లెక్కించడం.

ఈ DOC నుంచి - మరియు ఈ పది DOCS నుంచి - ఒక్క వాక్యం

ఒక function ఏదో ఒకటి తిరిగి ఇస్తే అది పనిచేసినట్టు కాదు. అది ఏమీ చెప్పకుండా తిరిగి వచ్చినట్టు.

§4 lo true వచ్చింది · §7 lo సరైన జవాబే వచ్చింది · §10 lo ఒక చెల్లుబాటు అయ్యే node వచ్చింది.

మూడింటిలోనూ code విఫలం కాలేదు - నేను తప్పు ప్రశ్న అడిగాను.

కాబట్టి ప్రతి తనిఖీ తర్వాత ఒక తనిఖీ: "ఈ పరీక్ష విఫలం కాగలదా?" అదే mutation testing, మరియు అదే ఈ పది docs lo ప్రతి "0 ఉల్లంఘనలు" వెనక ఉన్నది.